

TR IDSIA-19-03, Version 5
December 2006, arXiv:cs.LO/0309048 v5
(v1: 25 September 2003)

哥德尔机：可证明最优自我 改进的自指通用问题求解器

于尔根·施密德胡贝尔 IDSIA, Galleria 2, 6928 Manno-Lugano, 瑞士
& 慕尼黑工业大学, Boltzmannstr. 3, 85748 Garching, 慕尼黑, 德国
juergen@idsia.ch - <http://www.idsia.ch/~juergen>

摘要

本文提出第一类数学上严格、通用、完全自指、自我改进且最优高效的问题求解器。受库尔特·哥德尔著名的自指公式（1931 年）启发，此类问题求解器一旦找到证明表明重写自身代码的任意部分是有用的，便立即执行重写，其中问题相关的效用函数、硬件以及整个初始代码由编码在初始证明搜索器中的公理所描述，该搜索器也是初始代码的一部分。搜索器以渐进最优高效的方式系统地测试可计算的证明技术（输出为证明的程序），直到找到一个可证明有用且可计算的自重写。我们证明这种自重写是全局最优的——不存在局部极大值！——因为代码首先必须证明继续搜索替代自重写的证明是无用的。与 Hutter 先前基于硬编码证明搜索器的非自指方法不同，我们的方法不仅具有最优的复杂度阶，而且能够最优地减少 $O()$ 符号所隐藏的任何减速，前提是这种加速的效用是可证明的。

关键词：自指，强化学习，问题求解，证明技术，最优通用搜索，自我改进

哥德尔机出版物 截至 2006 年（库尔特·哥德尔诞辰 100 周年，及其奠定理论计算机科学基础的里程碑论文 [11] 发表 75 周年）：[45, 47, 48, 49, 50, 51]。

目录

1 引言与大纲 2

2 概述 / 基本思想 / 局限性	2
2.1 设定与形式目标	3
2.2 哥德尔机的基本思想	4
2.3 证明技术与一个 $O()$ 最优初始证明搜索器	4
2.4 哥德尔机的局限性	5

3 一个代表性哥德尔机的基本细节	6
3.1 证明技术	6
3.2 证明技术使用的重要指令	6

4 全局最优性定理	11
4.1 给定 u 和 $\mathcal{A}$ 编码于 p 中的全局最优自修改	11
4.2 替代的松弛目标定理	12
4.3 全局最优性与递归元层级	12
4.4 证明目标定理有多困难?	12

5 偏差最优证明搜索 (Bias-Optimal Proof Search, BIOPS)	13
5.1 证明空间中的在线通用搜索	13

5.2 存活的证明搜索器如何利用最优有序问题求解器来

解决剩余的证明搜索任务	14
-------------	----

6 讨论与先前工作	15
6.1 哥德尔机自我改进的可能类型	15
6.2 应用示例	17
6.3 概率哥德尔机硬件	17
6.4 与先前工作的关系	18
6.5 人类是概率哥德尔机吗?	21
6.6 哥德尔机与意识	22
6.7 常见问题	22

7 结论 24

8 致谢 24

1 引言与概述

所有用于问题求解、机器学习、强化学习的传统算法 [20] 都是硬连线的。其中一些被设计为通过经验改进某种有限类型的策略，但它们不属于可修改策略的一部分，也无法以理论上可靠的方式自我改进。需要人类来创造新的或更好的问题求解算法，并在适当假设下证明其有效性。本文将尽可能普遍地消除对人类努力的这一限制性需求，将所有工作（包括证明搜索）留给一个能够以任意可计算方式、以最高效方式重写并改进自身的系统。为攻克这一“人工智能的重大问题”[46]，我们引入一类新颖的最优、完全自指 [11] 的通用问题求解器，称为哥德尔机 [45, 47, 49, 51, 50]。¹ 它们是通用问题求解系统，与某个（部分可观测的）环境交互，原则上可以无本质限制地修改自身，仅受可计算性限制。其初始算法并非硬连线；它可以完全重写自身，但前提是嵌入初始算法中的证明搜索器能够首先证明该重写是有用的，给定一个形式化的效用函数，该函数反映计算时间和预期的未来成功（例如奖励）。我们将看到，由此方法产生的自重写实际上是全局最优的（定理 4.1，第 4 节），相对于哥德尔关于可证明性的著名基本限制 [11]。这些限制不应令我们担忧；如果某个自重写的效用无法被证明，那么人类也几乎无能为力。初始证明搜索器在定理 5.1（第 5 节）的意义下是 $O()$ -最优的（具有最优复杂度阶）。然而，与硬连线系统（如 Hutter 的 [16, 17] 和 Levin 的 [24, 26]，第 6.4 节）不同，哥德尔机原则上可以加速其初始软件的任何部分，包括其证明搜索器，以满足超出 $O()$ -记号所能表达的任意可形式化的最优性概念。本文方法产生了首个理论上可靠、完全自指、最优的通用问题求解器。

概述。第 2 节介绍基本概念与根本局限性，第 3 节阐述自指公理系统的核心细节，第 4 节给出全局最优性定理 4.1，第 5 节描述 $O()$ -最优（定理 5.1）初始证明搜索器。第 6 节提供示例及与先前工作的关联，简要讨论诸如意识的技术性论证等问题，并列出关于哥德尔机的若干常见问题解答。

2 概述 / 基本思想 / 局限性

计算机科学的许多传统问题在问题求解过程开始时仅需一个定义问题的输入。例如，初始输入可能是一个大整数，目标可能是对其进行因式分解。然而，在下文中，我们还将考虑更一般的情形，即问题求解需要与动态的、初始未知的环境进行交互，该环境持续产生输入流和反馈信号，例如在自主机器人控制任务中，目标可能是最大化期望累积未来奖励 [20]。这

¹ 或写作‘Goedel machine’，以避免变音符号。但‘Godel machine’并不完全正确。勿与彭罗斯在不同语境下所称的‘哥德尔假想定理证明机’[30]相混淆！

可能需要求解本质上任意的问题（第 6.2 节中的示例将传统问题表述为特例）。

2.1 设定与形式目标

我们的硬件（例如，通用或空间有界的图灵机 [61] 或个人计算机的抽象模型）具有单一的生命周期，由离散的周期或时间步组成 $t = 1, 2, \dots$ 。其总寿命 T 可能事先已知，也可能未知。在下文中，任何时变变量 Q 在时间 t 的值将用 $Q(t)$ 表示。在每个周期中，我们的硬件执行一个基本操作，该操作影响其变量状态 $s \in \mathcal{S} \subset \mathcal{B}^*$ （不失一般性， $\mathcal{B}^*$ 是二进制字母表 $B = \{0, 1\}$ 上所有可能位串的集合），并且可能还影响变量环境状态 $Env \in \mathcal{E}$ （这里我们尚不需要指定与问题相关的集合 $\mathcal{E}$ ）。存在一个硬连线的状态转移函数 $F : \mathcal{S} \times \mathcal{E} \rightarrow \mathcal{S}$ 。对于 $t > 1$, $s(t) = F(s(t-1), Env(t-1))$ ，状态处于周期 $t-1$ 的硬件操作已完成、但周期 t 的操作尚未开始的时刻。 $Env(t)$ 可能依赖于编码在 $s(t-1)$ 中的过去输出动作，并由可能具有反应性的环境同时更新或（概率性地）计算。为了方便讨论程序和数据，我们经常为某些字符串变量命名，这些变量编码为 s 的组件或子串。特别值得关注的是三个变量，分别称为时间 x, y 和 p ：1. 在时间 t ，变量 $time$ 保存 t 的唯一二进制表示。我们将 $time(1) = \text{初始化为 '1'}$ ，即仅由一个 1 组成的位串。硬件从一个周期到下一个周期递增 $time$ 。这最多需要 $O(\log t)$ 步，平均仅需 $O(1)$ 步计算。

2. 变量 x 保存从环境输入到哥德尔机的数据。对于 $t > 1, x(t)$ ，只有当运行在哥德尔机上的程序在时间 $t-1$ 执行了特殊的输入请求指令时，才可能与 $x(t-1)$ 不同。一般而言，连续输入之间的延迟应足够大，以便程序能够对输入执行某些基本计算，例如在下一个输入到达之前将其复制到内部存储（ s 的保留部分）中。

3. 变量 y 保存哥德尔机的输出。 $y(t)$ 是一个输出位串，随后可能影响环境，其中默认情况下为 $y(1) = \text{'0'}$ 。例如， $y(t)$ 可以被解释为环境操纵机器人的控制信号，其动作可能对未来输入产生影响。4. $p(1)$ 是初始软件：一个实现与环境交互的原始（次优）策略的程序，表示为 $p(1)$ 的子串 $e(1)$ ，加上用于搜索证明的原始策略。细节将在下文讨论。

At any given time t ($1 \leq t \leq T$) the goal is to maximize future success or *utility*. A typical “value to go” utility function is of the form $u(s, Env) : \mathcal{S} \times \mathcal{E} \rightarrow \mathcal{R}$, where $\mathcal{R}$ is the set of real numbers:

$$u(s, Env) = E_\mu \left[\sum_{\tau=time}^T r(\tau) \mid s, Env \right], \quad (1)$$

其中 $r(t)$ 是时间 t , $E_u(\cdot | \cdot)$ 处的实值奖励输入（编码在 $s(t)$ 内），表示关于某个可能未知分布 μ 的条件期望算子，该分布来自可能分布集合 M （ M 反映了关于环境可能概率反应的所有已知信息），而上述 $time = time(s)$ 是状态 s 的函数，唯一标识当前周期。注意，我们考虑了通过适当行动延长预期寿命的可能性。其他可形式化的效用函数可以倾向于改善最坏情况而非期望未来性能，或提高单位时间内的奖励获取等。显然，计算机科学的大多数经典问题都可以在此框架中表述——参见第 6.2 节中的示例。

2.2 哥德尔机的基本思想

我们的机器通过加载一种特定形式的机器相关、自修改代码 p ，成为一台自指涉的 [11] 哥德尔机。时间步 1 处的初始代码 $p(1)$ 包括一个（通常次优的）用于与环境交互的问题求解子程序 $e(1)$ ，例如何传统的强化学习算法 [20]，以及一个通用证明搜索子程序（第 5 节），该子程序系统地生成（切换程序，证明）对 s 的可变子串，直到找到目标定理的证明，该定理本质上陈述：“在当前机器上通过当前程序切换程序立即重写 p ，比保持 p 不变带来更高的效用”。然后执行切换程序，这可能完全重写 p ，包括证明搜索器。第 3 节将解释编码在 $p(1)$ 中的必要初始公理系统 $\mathcal{A}$ 的细节。对比图 1。全局最优性定理（定理 4.1，第 4 节）表明这种自我改进策略并非贪婪：由于“保持 p 不变”的效用隐式评估了未修改的 p 以后可能找到的所有可能的替代切换程序，我们获得了全局最优的自我改变——当前的切换程序代表了所有可能的相关自我改变中的最佳选择，相对于给定的资源限制和初始证明搜索策略。

2.3 证明技术与一个 $O()$ -最优初始证明搜索器

第 5 节将给出证明搜索器的一个 $O()$ 最优初始化，即具有最优复杂度阶的初始化（定理 5.1）。然而，仍将存在大量被 $O()$ 记号所掩盖的自我改进空间。该搜索器使用通用搜索 [24, 26] 的在线扩展来系统地测试在线证明技术，这些证明技术是生成证明的程序，可以读取状态 s 的部分内容（类似地，数学家通常对证明技术比对定理更感兴趣）。为了证明上述目标定理，证明技术可以调用特殊指令来生成公理并应用推理规则，以通过定理延长当前证明。这里，编码在 $p(1)$ 中的公理系统 $\mathcal{A}$ 包含描述以下内容的公理：（a）在给定硬件上运行的程序所调用的任何指令如何将机器状态 s （包括指令指针等）从一个步骤改变到下一个步骤（这样证明技术就可以推理包括证明搜索器在内的任何程序的效果），（b）初始程序 $p(1)$ 本身（第 3 节将证明这可以在不引入循环性的情况下实现），（c）随机环境属性，（d）形式效用函数 u ，例如方程（1），它考虑了包括证明搜索在内的所有动作的计算成本。

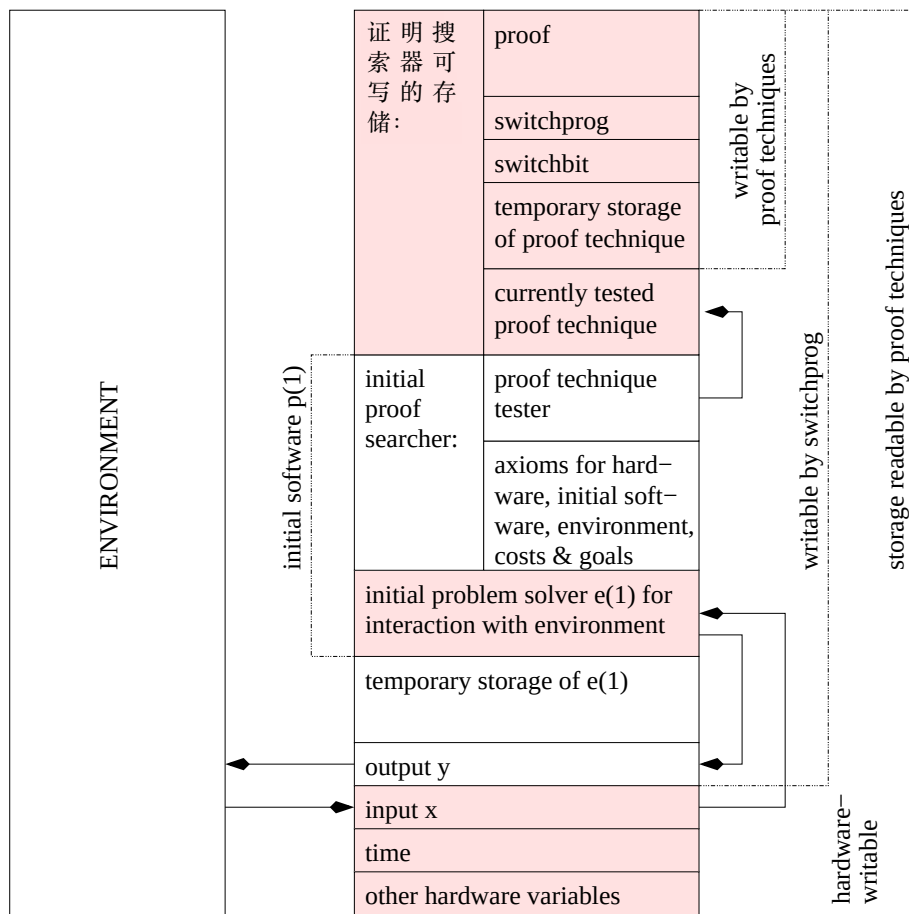


图 1：一个尚未自我改进的示例哥德尔机的存储快照，初始软件仍然完好。详见正文。

2.4 哥德尔机的局限性

这些根本性局限与哥德尔（Gödel）关于自指公式的著名论文中首次指出的局限密切相关 [11]。任何涵盖算术（或策梅洛-弗兰克尔集合论等）的形式系统，要么存在缺陷，要么允许存在不可证明但为真的命题。因此，即便是拥有无限计算资源的哥德尔机，也必须忽略那些其有效性无法证明的自我改进，例如由于 $\mathcal{A}$ 中缺乏足够强大的公理。特别地，可以构造出环境与效用函数的病态示例，使得机器永远无法证明某个目标定理。可对比基于某些不可计算谓词的布鲁姆加速定理 [3, 4]。类似地，资源有限的现实哥德尔机无法从那些在其时间与空间约束内无法证明其有用性的自我改进中获益。然而，与以往方法不同，它原则上至少可以利用其初始软件任何部分中可证明有益的加速，包括那些负责巨大（但与问题类别无关）减速的部分，而早期方法忽略了这些部分 [16, 17]（第 6.4 节）。

3 一台代表性哥德尔机的基本细节

记号。除非另有说明或显而易见，本文中引入的新变量和函数均假定覆盖上下文中隐含的取值范围。 $l(q)$ 表示比特串 q ; q_n 的位数， n 的第 q ; λ 位，空串（其中 $l(\lambda) = 0$); $q_{m:n} = \lambda$ 若 $m > n$

and $q_m q_{m+1} \dots q_n$ rwi (where $q_0 := q_{0:0} := \lambda$).

定理证明需要一个公理模式，该模式给出形式逻辑系统 $\mathcal{A}$ 的可枚举公理集，其公式和定理是某个有限字母表上的符号串，该字母表可包含传统逻辑符号（如

$\rightarrow, \wedge, =, (,), \forall, \exists, \dots, c_1, c_2, \dots, f_1, f_2, \dots,)$ 、概率论符号（如 $E(\cdot)$ ，期望算子）、算术符号（ $+, -, /, =, \sum, <, \dots,)$ ）、字符串操作符号（特别是用于表示任意时刻状态 s 任意部分的符号，如 $s_{7.88}(5555)$ ）。证明是一个定理序列，其中每个定理要么是公理，要么是通过应用推理规则（如与合一结合的肯定前件式，例如 [10]）从先前定理推导得出。本文其余部分将省略任何证明论教材中可找到的标准知识。我们不会冗长地列出特定 $\mathcal{A}$ 的所有公理，而是聚焦于新颖且关键的细节：如何克服自指带来的潜在问题，以及如何处理可能微妙的在线证明生成，这些证明讨论并影响当前正在运行的证明生成器本身。

3.1 证明技术

暴力穷举证明搜索器（用于胡特的工作中）[16, 17]；综述见第 6.4 节）按证明规模顺序系统地生成所有证明。为产生某一特定证明，所需时间随证明规模呈指数增长。而本文 $O()$ -最优 $p(1)$ 将产生许多算法复杂度较低的证明 [57, 22, 27] 快得多。它系统地测试（见第 5 节）被称为证明技术的程序，这些程序用通用语言编写 $\mathcal{L}$ 在内部实现 $p(1)$ 。例如， $\mathcal{L}$ 可能是 PROLOG 的一种变体 [7] 或通用 FORTH [29] 受近期最优搜索研究启发而设计的编程语言 [48] 一种证明技术由若干指令组成，这些指令允许任何部分 s 待读取的内容，例如以变量编码的输入 x （的子串 s ）或代码 $p(1)$ 。它可能会写入 s^p ，的一部分 s 保留用于临时结果。它还可以重写切换程序，并生成一个逐步增长的证明，该证明放置在字符串变量证明中，存储于 s 。证明和 s^p 在每次新的证明技术测试开始时被重置为空字符串。除了标准的算术和函数定义指令 [48] 修改 s^p 之外，编程语言 $\mathcal{L}$ 包含特殊指令（详见第 3.2 节），用于通过正确的定理延长当前证明、设置切换程序，以及检查是否找到了可证明最优的 p 修改程序并应立即执行。某些长证明可以由短证明技术产生。

3.2 证明技术使用的重要指令

以下六个证明修改指令（没有其他指令）的性质使得不可能将错误定理插入证明中，从而使证明验证变得简单。首先提供一个

最重要指令的简要概述：**get-axiom(n)** 将第 n 个可能的公理追加到当前证明中，**apply-rule(k, m, n)** 将第 k 条推理规则应用于当前证明中的第 m 和第 n 个定理（并追加结果），**set-switchprog(m,n)** 设置 $switchprog := s_{m,n}^p$ ，**check()** 测试证明中的最后一个定理是否是一个目标定理，表明通过 switchprog 进行自重写是有用的。细节如下。

1. 获取公理（get-axiom）函数以一个整数 n 作为参数，该整数由当前正在测试的证明技术的前缀借助算术指令计算得出，这些算术指令与先前工作中使用的类似 [48]。然后，该函数将第 n 条公理（如果根据下述公理方案存在）作为定理追加到证明中当前的定理序列中。初始公理方案编码如下：

(a) 硬件公理描述硬件，正式规定 s 的某些组件（环境输入 x 除外）如何从一个周期变化到下一个周期。例如，如果硬件是图灵机 2 (TM) [61]，则 $s(t)$ 是一个比特串，编码图灵机所有磁带的当前内容、其扫描头的位置以及图灵机有限状态自动机的当前内部状态，而 F 则规定图灵机的查找表，该查找表将内部状态与扫描头上方比特的任意可能组合映射到一个新的内部状态和一个动作，例如：将某个磁头的当前比特替换为 1/0，将某个扫描头递增（右移）或递减（左移），读取下一个输入比特并将其复制到输入磁带扫描头上方的单元中，等等。或者，如果硬件由具有有限存储的现代微处理器的抽象模型给出，则 $s(t)$ 将编码当前存储内容、寄存器值、指令指针等。例如，以下公理可以描述存储在 $s_{1:64}$ 中的某个 64 位硬件的指令指针如何在无溢出且 s_{65} 的值不指示应跳转到其他地址的情况下持续递增： $(\forall t \forall n : [(n < 2^{64} - 1) \wedge (n > 0) \wedge (t > 1) \wedge (t < T)])$

$$\begin{aligned} & \wedge (string2num(s_{1:64}(t)) = n) \wedge (s_{65}(t) = '0') \\ & \rightarrow (string2num(s_{1:64}(t+1)) = n+1) \end{aligned}$$

此处所用符号的语义，如“ $($ ”和“ $>$ ”以及“ $\rightarrow$ ”（蕴含），均为传统语义，而“ $string2num$ ”表示一个将比特串转换为数字的函数。显然，任何抽象硬件模型都可以用类似的方式完全公理化。

(b) 奖励公理定义了任何硬件指令的计算成本，以及输出动作的物理成本，例如编码在 $s(t)$ 中的控制信号 $y(t)$ 。相关公理为某些输入事件（编码在变量 x 中，是 s 的子串）赋予代表奖励或惩罚的值（例如，当哥德尔机控制的机器人撞到障碍物时）。图灵将哥德尔不可证明性结果用图灵机 (TMs) [61] 重新表述，图灵机随后成为最广泛使用的抽象计算模型。众所周知，存在通用图灵机，在某种意义上可以模拟任何其他图灵机或任何其他已知计算机。哥德尔基于整数的形式语言可以用来描述任何通用图灵机，反之亦然。

额外的公理将哥德尔机生命的总价值定义为所有奖励（例如，它们的总和）和在第 1 周期与 T 周期之间经历的成本的标量值函数。例如，假设 $s_{17:18}$ 只能通过外部输入改变；以下示例公理表明，每当这样的输入等于‘11’时，总奖励增加 3（未解释的符号具有显而易见的含义）：

$$(\forall t_1 \forall t_2 : [(t_1 < t_2) \wedge (t_1 \geq 1) \wedge (t_2 \leq T) \wedge (s_{17:18}(t_2) = '11')]) \\ \rightarrow [R(t_1, t_2) = R(t_1, t_2 - 1) + 3],$$

其中 $R(t_1, t_2)$ 被解释为时间 t_1 和 t_2 之间的累积奖励。显然，任何产生奖励的形式化方案都可以用类似的方式完全公理化。

(c) 环境公理限制了环境对输出序列 y （编码在 s 中）的反应方式，即产生新输入（编码在 s 的某些子串中）的方式。例如，可以预先知道环境是从给定可能分布集合 M 中包含的未知概率分布 μ 中采样的（比较方程 1）。例如， M 可以包含所有可计算的分布，给定先前历史 [57, 58, 16]，或者至少是极限可计算的 [41, 42]。或者，更严格地说，环境可能是某个未知但确定性的计算机程序 [63, 39]，从速度先验 [43] 中采样，该先验对任何方法都难以计算的环境赋予低概率。或者与环境的接口是马尔可夫的 [35]，即当前输入总是唯一地标识环境状态——在这个特殊情况下已经做了大量工作 [33, 2, 60]。甚至更严格地说，环境可能以完全可预测的、预先已知的方式演化。所有这些先验假设都可以在适当的 $\mathcal{A}$ 中完美形式化（否则我们无法撰写关于它们的科学论文）。

(d) 不确定性公理；字符串操作公理： 算术、微积分、概率论 [21]、统计学以及字符串操作的标准公理（结合硬件公理和环境公理）允许构造关于 $s(t)$ 未来值的（可能不确定的）性质的证明，以及在给定时间 τ 和 $s(\tau)$ 分量的某些假设值等条件下，关于期望剩余寿命/成本/奖励的界。一个示例定理，说明关于未来输入 x 的期望性质，可能如下所示：

$$(\forall t_1 \forall \mu \in M : [(1 \leq t_1) \wedge (t_1 + 15597 < T) \wedge (s_{5:9}(t_1) = '01011') \\ \wedge (x_{40:44}(t_1) = '00000')] \rightarrow (\exists t : [(t_1 < t < t_1 + 15597) \\ \wedge (P_\mu(x_{17:22}(t) = '011011' \mid s(t_1)) > \frac{998}{1000})])),$$

其中 $P_\mu(\cdot \mid \cdot)$ 表示相对于由环境公理（第 1c 项）描述的一组分布 M 中的公理化先验分布 μ 的条件概率。给定特定的可形式化硬件（第 1a 项）和关于可能概率性环境（第 1c 项）的可形式化假设，显然可以完全公理化

基于证明的关于过去和未来机器状态推理所需的一切。

(e) 初始状态公理：关于如何重建初始状态 $s(1)$ 或其部分的信息，使得证明搜索器能够构建包含以下类型公理的证明

$$(s_{\mathbf{m:n}}(1) = \mathbf{z}), \text{ e.g. : } (s_{7:9}(1) = '010').$$

此处及本文其余部分，我们在公式中使用粗体表示语法占位符（如 $\mathbf{m,n,z}$ ），代表符号串变量（如 $\mathbf{m,n,z}$ ），其语义在正文中解释——在当前上下文中 z 是位串 $s_{\mathbf{m:n}}(1)$ 。注意，将硬件描述和初始硬件描述 p 完全编码在 p 自身内部并非根本性问题。为理解这一点，观察到某些软件可能包含一个可以打印该软件的程序。

(f) 效用公理，以效用函数 u 的形式描述总体目标；例如，第 2.1 节中的方程 (1)。

2. `apply-rule(k, m, n)` 以推理规则（如假言推理，存储在 $p(1)$ 内编码的可能推理规则列表中）的索引 k （如果存在）以及当前证明中两个先前已证明定理（按其创建顺序编号）的索引 m, n 作为参数。如果适用，则将相应的推理规则应用于所指向的定理，并将所得定理附加到证明中。否则，当前测试的证明技术被中断。这确保了证明永远不会被输入无效证明。

3. 删除定理 (`delete-theorem`) (m) 删除当前存储证明中的第 m 条定理，从而释放存储空间，使证明存储部分 s 可被重用，且最大证明规模不必受存储约束的限制。然而，从证明中删除的定理无法再被应用规则 (`apply-rule`) 寻址，以产生证明的进一步延伸。

4. 设置切换程序 (`set-switchprog`) (m,n) 将 `switchprog` 替换为 $s_{m:n}^p$ ，前提是 $s_{m:n}^p$ 确实是 s^p 的非空子串，即证明技术可写的存储。

5. 检查 (`check`) () 验证证明搜索的目标是否已达到。首先测试证明中最后一条定理（若有）是否具有目标定理的形式。目标定理陈述：给定当前公理化的效用函数 u （第 1f 项），从 p 切换到当前切换程序的效用将高于继续执行 p （后者将继续搜索替代切换程序）的效用。目标定理是符号串（以 s 编码），其（解码后的）形式为

$$\begin{aligned} & (u[s(\mathbf{t_1}) \oplus (\text{switchbit}(\mathbf{t_1}) = '1'), \text{Env}(\mathbf{t_1})] > \\ & u[s(\mathbf{t_1}) \oplus (\text{switchbit}(\mathbf{t_1}) = '0'), \text{Env}(\mathbf{t_1})]) \end{aligned} \quad (2)$$

其中变量 t_1 （由语法占位符 $\mathbf{t_1}$ 表示）代表一个时间步，而所有其他符号属于定理证明演算的字母表，即可能的目标定理集合仅由 t_1 参数化。此处演算应允许

记号 $s(t_1) \oplus (\text{switchbit}(t_1) = 'b')$ 作为状态的简写，该状态是在时刻 t_1 将变量位 switchbit （编码于 s 中）的真值替换为 $b \in \{0,1\}$ 、并替换开关位 (t_1 后得到的。这将便于表述那些以 $s(t_1)$ 的各种备选假设性质为条件来比较取值的定理。（注意，即使在 $s(t_1)$ 和开关位 (t_1) 对所有有效 t_1 都完全预先确定的环境中，当前证明技术也可能仅部分知晓 $s(t_1)$ ）。引入 t_1 的目的是处理检查与切换中可能涉及的硬件特定时间延迟——将证明搜索期间找到的抽象符号串与哥德尔机的真实当前状态进行匹配可能需要相当长的时间。若已找到目标定理，`check()` 使用一个简单的预置子程序（当然也编码在 $p(1)$ 中）来检查是否有足够时间在持续增长的 time 等于 t_1 之前将变量开关位（初始为 0）置为 1。若该子程序返回否定结果，`check()` 退出。否则它将开关位置为 $:= 1$ （没有其他改变开关位的方式）。然后它反复测试 time 直到 $\text{time} > t_1$ ，以确保公式 (2) 的条件在 t_1 处得到满足。随后它将控制权转移给 `switchprog`（没有其他调用 `switchprog` 的方式）。`switchprog` 随后可以重写 s 的所有部分，但不包括硬件保留部分如 time 和 x ，而包括 p 。

6. `state2theorem(m, n)` 接受两个整数参数 m, n ，并尝试将 $s_{m:n}$ 的当前内容转换为如下形式的定理

$$(s_{m:n}(t_1) = z), \text{ e.g. : } (s_{6:9}(7775555) = '1001'),$$

其中 t_1 表示在 `state2theorem` 被调用后不久（通过检查时间）测得的时间， z 表示位串 $s_{m:n}(t_1)$ （回忆第 1e 项的特殊情形 $t_1 = 1$ ）。因此，我们接受 s 任意部分带时间标记的当前可观测内容作为定理，而无需从初始状态 $s(1)$ 等出发以其他方式证明，因为迄今为止的计算已经证明了该定理为真。这样，我们可以利用环境输入所传达的信息，以及这样一个事实：有时（但并非总是）确定程序输出的最快方法就是运行它。

然而，这种语法与语义之间的非传统在线接口需要特别小心。我们必须避免通过 s 中在读取过程中发生变化的部分产生不一致的结果。例如，快速变化的指令指针 IP（由硬件持续更新）的当前值可能基本上无法读取，因为读取子程序本身的执行就会多次修改 IP。为了方便起见，可以设置（通常有限的）硬件，使其每 c 个周期将快速硬件变量的内容存储在 s 的保留部分中，这样 `state2theorem()` 的适当变体至少可以将某些最近的快速变量值转换为定理。然而，这不会消除与自观察相关的所有问题。例如，要读取的 $s_{m:n}$ 可能还包含读取过程自身的、临时的、不断变化的字符串指针变量等。³ 为了解决内存有限的计算机上的这些问题，我们看到当前 s 的某些部分可能无法在不改变可观察对象本身的情况下直接观察。然而，有时公理和先前的观察将允许哥德尔机推断出不可直接观察的随时间变化的存储内容。例如，通过分析上述示例中通过指令指针 IP 执行的代码，IP 在某些时刻的值可能是可预测的（或事后可追溯的）。

state2theorem 首先使用某个固定协议（当然编码在 $p(1)$ 中）来检查当前 $s_{m:n}$ 是否可读，或者如果由 state2theorem 的剩余代码读取它是否会发生变化。如果 *If so, or if m, n , 不在适当范围内*，则该指令没有进一步的效果。否则，它将一个形式为 $(s_{m:n}(t_1) = z)$ 的观察到的定理附加到证明中。例如，如果当前时间是 7770000，那么调用 state2theorem(6,9) 可能返回定理 $(s_{6:9}(7775555) = '1001')$ ，其中 $7775555 - 7770000 = 5555$ 反映了 state2theorem 执行初始检查并从持续增加的 *time* 中读取前导位所需的时间（读取 *time* 也需要时间），以便它可以确定 7775555 是 state2theorem 开始后的一个最近的适当时间标签。

公理系统 $\mathcal{A}$ 是给定哥德尔机的一个定义参数。显然， $\mathcal{A}$ 必须足够强大，以允许证明目标定理。特别是，不确定性公理理论（第 1d 项）必须足够丰富。这并非根本性问题：我们只需插入概率论的所有传统公理 [21]。

4 全局最优性定理

直观上，在任何给定时间 p ，只有当某个自修改算法（通过指令 check()——上述第 5 项）在所有可能的自修改中，根据效用函数（通常依赖于可用资源，如存储大小和剩余寿命）是“最佳”时，才应执行该算法。然而，乍一看，目标定理（2）似乎隐含地只涉及一个单一的修改算法，即由系统证明搜索器在时间 t_1 设置的 switchprog (t_1)。这种局部搜索不是贪心的吗？它不会导致局部最优而非全局最优吗？根据以下全局最优性定理，答案是否定的。

4.1 给定条件下的全局最优自修改

u and $\mathcal{A}$ Encoded in p

定理 4.1 给定任何可形式化的效用函数 u （第 1f 项），并假设底层形式系统 $\mathcal{A}$ 的一致性，通过执行某个由目标定理（2）的证明所识别的程序 switchprog 而获得的 p 的任何自修改，在以下意义上是全局最优的：开始执行当前 switchprog 的效用高于等待证明搜索器稍后产生替代 switchprog 的效用。

证明。 目标定理（2）隐含地涉及证明搜索器未来可能产生的所有其他 switchprog。要看到这一点，考虑二元决策的两个备选方案：（1）要么执行当前的 switchprog（设置 switchbit = 1），要么（2）继续搜索证明和 switchprog（设置 switchbit = 0），直到系统搜索器找到更好的 switchprog。显然，第二个备选方案涉及所有（可能无限多个）稍后要考虑的潜在 switchprog。也就是说，如果当前的 switchprog 不是“最佳”的，那么其他变量在给定时间的值可能根本无法推导。这种自观察的局限性让人想起海森堡著名的不确定性原理 [12]，该原理指出某些物理测量必然不精确，因为测量过程会影响被测量量。

证明搜索器将无法证明，在 $\mathcal{A}$. *Q.E.D.* 一致性的假设下，设置开关位并执行开关程序会比丢弃开关程序带来更高的期望奖励。第 5 节中的初始证明搜索器已经以 $O()$ 最优的方式生成了所有可能的证明和开关程序。然而，由于它是 $p(1)$ 的一部分，其证明可以涉及证明搜索器自身，可能触发证明搜索器重写，从而获得优于单纯 $O()$ 最优的性能。

4.2 替代的松弛目标定理

我们可以用以下替代目标定理替换目标定理 (2) (第 5 项)：

$$\begin{aligned} & (u[s(\mathbf{t}_1) \oplus (\text{switchbit}(\mathbf{t}_1) = '1'), Env(\mathbf{t}_1)] \geq \\ & u[s(\mathbf{t}_1) \oplus (\text{switchbit}(\mathbf{t}_1) = '0'), Env(\mathbf{t}_1)]) \end{aligned} \quad (3)$$

与原始目标定理 (2) 的唯一区别在于，“ $>$ ”符号变成了“ $\geq$ ”符号。也就是说，哥德尔机一旦找到证明表明该改变不会使情况变差，就会立即改变自身。类似于定理 4.1 的全局最优性定理成立；只需将定理 4.1 中的最后一句话替换为：启动当前开关程序执行的效用至少与等待证明搜索器稍后产生替代开关程序的效用一样高。

4.3 全局最优性与递归元层级

我们完全自指设置中最重要的方面之一如下。任何目标定理的证明都自动以递归方式证明了相应的自我修改对于当前修改所影响的所有进一步自我修改都是有益的。在这个意义上，自指系统所有可能的“元层级”都被折叠为一个。

4.4 证明目标定理有多难？

当然，这取决于任务和初始公理 $\mathcal{A}$ 。设计简单的任务和相应的一致 $\mathcal{A}$ ，使得目标定理存在简短而平凡的证明，是直截了当的。即使我们通过渐近最优且相当通用的方法（如 Hutter 的 $AIXI(t,l)$ [16, 19]）来初始化初始问题求解器 $e(1)$ ，也可能直接证明切换到另一种策略是有用的，尤其是当 $\mathcal{A}$ 包含超出 $AIXI(t,l)$ 所设公理假设的额外先验知识时。后者需要一个非常耗时但恒定的设置阶段，其成本在 $O()$ 记号中消失，但在如方程 (1) 的 u 这样的效用函数中不会消失。例如，只需构造一个环境，其中最大奖励通过执行一个永无止境的简单但有益的动作序列来实现，比如反复按压杠杆，再加上一个非常简单的公理系统 $\mathcal{A}$ ，该系统允许一个简短的证明，表明中断无奖励的设置阶段并开始按压杠杆是有用的。

另一方面，可以构造出无法证明目标定理的情形，例如利用不可判定性理论的结果，如 [11, 32, 3, 4]。特别地，采用赖斯定理 [32] 所体现的极端平凡性概念（一般函数上的任何非平凡性质都是不可判定的），只有对给定策略的平凡改进才可能被证明是有用的。然而，这种平凡性概念显然没有反映科学家直觉上所认为的平凡。尽管机器学习文献中的许多定理，以及更广泛的计算机科学文献中的定理，都局限于在赖斯意义下平凡的函数性质，但它们在直觉上被广泛认为是非平凡的。事实上，赖斯定理所涉及的函数类的无限域，不仅对大多数处理现实世界问题的科学家来说无关紧要，而且对典型的哥德尔机来说也是如此，因为哥德尔机在其有限的生命周期内只能处理有限数量的事件。一般而言，在明显平凡和明显非平凡的情形之间，存在许多不太明显的情形。关键在于：我们通常事先不知道是否有可能以可证明的良好方式改变给定的初始问题求解器。传统方法是投入人类研究努力去查明。然而，哥德尔机可以自行完成这一任务，除了可计算性和可证明性的限制外，没有本质上的限制。注意，为了证明目标定理，证明技术不一定需要计算切换和不切换的真实期望效用——它只需要确定哪一个更高。例如，可能很容易证明，将证明搜索器的某个子程序加速 2 倍，肯定值得执行子程序更改算法所需的（与生命周期 T 相比）可忽略的时间，无论切换的精确效用是多少。

5 偏差最优证明搜索 (BIOPS)

在此，我们构造一个初始 $p(1)$ ，它在下面将要描述的某种有限意义上是 $O()$ -最优的，但仍可能被改进，因为它在给定 u 的意义上不一定是最好的（例如，方程 (1) 的 u 既未提及也不关心 $O()$ -最优性）。我们的偏差最优证明搜索 (BIOPS) 本质上是通用搜索 [24, 26] 在证明搜索中的应用。然而，一个新颖之处在于：以往通用搜索的实用变体和扩展已被应用于 [38, 40, 55, 48] 离线程序搜索任务，其中程序输入是固定的，因此同一程序总是产生相同的结果。然而，在我们的在线设置中，BIOPS 必须考虑到，在不同时间启动的相同证明技术可能产生不同的证明，因为它可能读取 s 中随机器生命进程而变化的部分（例如，输入）。

5.1 证明空间中的在线通用搜索

BIOPS 从证明技术 w 上的概率分布 F （初始偏差）开始，这些证明技术可以在 $\mathcal{L}$ ，e.g., $F(w) = K^{-l(w)}$ 中为由 K 条可能指令 [26] 组成的程序编写。BIOPS 是近偏差最优的 [48]，即它不会在任何证明技术上花费比其根据概率偏差所应得的更多时间，即不超过其概率乘以总搜索时间：

定义 5.1（偏差最优搜索器 [48]）设 $\mathcal{R}$ 为问题类， $\mathcal{C}$ 为解候选的搜索空间（其中任何问题 $r \in \mathcal{R}$ 都应在 $\mathcal{C}$ ）， $F(q \mid r)$ 中有解，作为任务

以候选对象上的条件概率分布形式存在的依赖偏差 $q \in \mathcal{C}$ 。假设我们还有一个预定义的过程，可以在时间 $t(q, r)$ 内创建并测试任意给定的 q 在任意 $r \in \mathcal{R}$ 上（通常事先未知）。那么，一个搜索器是 n -偏差最优的（ $n \geq 1$ ），如果对于任意最大总搜索时间 $T_{total} > 0$ ，它保证能解决任何问题 $r \in \mathcal{R}$ ，只要该问题有一个解 $p \in \mathcal{C}$ 满足 $t(p, r) \leq P(p | r) T_{total}/n$ 。如果 $n = 1$ ，则它是偏差最优的。

方法 5.1 (BIOPS) 在阶段 $(i = 1, 2, 3, \dots)$ DO: F 或所有自定界 [26] 证明技术满足

$w \in \mathcal{L} \quad P(w) \geq 2^{-i}$ DO:

1. 运行 w 直至停机或出错（如除零）或消耗 $2^i P(w)$ 步。
2. 撤销 w 对 s^p 的影响（其耗时不会显著超过执行 w 。）

证明技术 w 只能通过调用指令检查（check()）（第 5 项）来中断方法 5.1，该调用可能将控制权转移给切换程序（switchprog），后者甚至可能删除或重写方法 5.1。由于初始 p 在形式化硬件上运行，并且由 p 测试的证明技术能够读取 p 以及 s 的其他部分，因此它们可以生成关于 p 和 BIOPS 本身（预期）性能的证明。方法 5.1 至少在以下意义上具有最优的计算复杂度阶。

定理 5.1 若独立于可变时间，某种未知的快速证明技术

w 至多需

要 $f(k)$ 步来生成难度测度 k （一个取决于待解决任务性质的整数）的证明，则方法 5.1 至多需要

$O(f(k))$ 步。

证明。 显而易见，方法 5.1 至多需要 $O(f(k)/P(w)) = O(f(k))$ 步——常数因子 $1/P(w)$ 不依赖于 k 。Q.E.D.。初始证明搜索本身仅为 $O()$ -最优。然而，再次注意，证明本身可能涉及相当不同的、任意可形式化的最优性概念（强于 $O()$ -记号中可表达的概念），这些概念由给定的、问题特定的、形式化的效用函数 u 所体现，特别是方程（1）意义下的最大未来奖励。这可能促使对初始证明搜索器进行有用的、影响常数的重写，尽管其 $O()$ -最优性概念有限（但流行且广泛使用）。一旦在哥德尔机总寿命的某个初始阶段之后找到并执行了有用的重写， $O()$ -最优性的限制便不再成为问题。

5.2 存活的证明搜索器如何利用最优有序概率

引理求解器以解决剩余证明搜索任务

以下内容对本文并非必需。假设与第一个找到的目标定理对应的 switchprog 的执行没有重写 p 自身的代码——当前的 p 仍等于 $p(1)$ ——并且已重置 switchbit 并将控制权返回给 p ，使其能够从被中断处继续。在这种情况下， $p(1)$ 的 BIOPS 子程序可以使用最优有序问题求解器 OOPS [48] 来加速对第 n 个目标定理（ $n > 1$ ）的搜索，方法是在可能的情况下重用先前找到的目标定理的证明技术。基本思想如下（细节：[48]）。

每当目标定理被证明时， $p(1)$ 会冻结相应的证明技术：它变得不可被后续证明搜索任务中待测试的证明技术写入，但仍保持可读，因此未来的证明技术可以对其进行复制编辑和/或将其作为子程序调用。我们还允许证明技术的前缀临时重写其后缀上的概率分布 [48]，从而本质上基于先前经验重写基于概率的搜索过程（方法 5.1 的增量扩展）。作为副作用，我们元搜索更快的搜索过程，这可以极大地加速新任务的学习 [48]。给定一个新的证明搜索任务，BIOPS 执行 OOPS，将总搜索时间的一半用于方法 5.1 的一个变体，该变体仅在以最近冻结的证明技术开头的自定义 [25, 6] 证明技术中进行搜索。其余时间用于具有任意前缀的全新证明技术（尽管可以重用先前冻结的证明技术）[48]。（我们也可以搜索一个泛化证明技术来解决迄今为止所有的证明搜索任务。在搜索的前半部分，我们不必在除最近任务之外的其他任务上测试证明技术，因为我们已经知道它们的前缀解决了先前的任务 [48]。）可以证明，给定初始偏差或由于先前任务的冻结解而产生的中间偏差，OOPS 本质上是 8-偏差最优的（见定义 5.1）[48]。这一结果立即适用于 BIOPS。总之，BIOPS 本质上将新任务总搜索时间的一部分分配给以可计算方式利用先前成功证明技术的证明技术。如果通过复制编辑/调用先前冻结的证明技术比从头解决新的证明搜索任务更快，那么 BIOPS 将发现这一点并从中获益。否则，至少它不会因先前的解决方案而显著减慢——BIOPS 将保持 8-偏差最优。然而，回想一下，BIOPS 并不是初始化哥德尔机证明搜索器的唯一可能方式。全局最优性定理 4.1（第 4 节）表达了相对于我们选择的任何初始证明搜索器的最优性。

6 讨论与先前工作

这里我们列举一些可能的自我改进类型（第 6.1 节）、哥德尔机在各种效用函数和环境定义的各种任务中的适用性（第 6.2 节）、概率硬件（第 6.3 节）以及与先前工作的关系（第 6.4 节）。我们还简要讨论了自指与意识（第 6.6 节），并提供了一份常见问题解答列表（第 6.7 节）。

6.1 哥德尔机自我改进的可能类型

哪些可证明有用的自我修改是可能的？哥德尔机能做的事情几乎没有限制。

1. 在最简单的情况下，它可能保持其基本证明搜索器不变，仅改变证明搜索子程序与子策略 e 之间的时间共享比例—— p 中负责与环境交互的部分。

2. 或者哥德尔机可能只修改 e 。例如，初始的 $e(1)$ 可能是一个定期在 s 某处存储有限过去事件记忆的程序；这可能使 p 推导出修改 e 是有用的，使得 e 将进行某些实验以增加关于环境的知识，并利用所得信息增加奖励获取。在这个意义上，哥德尔机体现了一种处理探索与利用问题 [20] 的原则性方法。注意，进行某项实验的期望效用（方程（1））可能超过不进行该实验的期望效用，即使实验结果表明应继续按照先前的 e 行动。

3. 哥德尔机也可能修改其公理本身以加快速度。例如，它可能找到一个证明，表明原始公理应被由原始公理推导出的定理所替换或扩充。

4. 哥德尔机甚至可能改变自己的效用函数和目标定理，但只有在根据旧值可证明新值更优时才能这样做。

5. 在许多情况下，我们不期望哥德尔机用完全放弃证明搜索的代码来替换其证明搜索器。相反，我们期望只有证明搜索器的某些子程序会被加速——比较第 4.4 节中的例子——或者可能只是生成证明的顺序会以特定于问题的方式被修改。这可以通过修改第 5 节中初始偏置最优证明搜索器的证明技术上的概率分布来实现。

6. 一般而言，有限重写的效用往往比完全重写的效用更容易证明。例如，假设现在是晚上 8 点，我们的哥德尔机（Gödel Machine）控制智能体的永久目标是通过（替代）目标定理（3）最大化未来期望奖励。其中一部分是避免饥饿。冰箱里没有食物，商店在晚上 8:30 关门。它没有时间在每一个细节上优化去超市的路线，但如果它现在不立即出发，今晚就会挨饿（原则上，这种行动的近期后果应该很容易证明，甚至可能以类似于人类向自己证明潜在行动优势的方式）。也就是说，如果智能体之前的策略尚未包含例如每天傍晚自动去超市的行程，那么该策略应当被证明至少以非常有限且简单的方式立即重写，趁还有时间，确保智能体今晚一定能获得一些食物，同时不影响那些可以稍后优化或决定的较不紧迫的未来行为，例如前往食物的路线细节或明天的行动。

7. 在某些无趣的环境中，奖励最大化是通过变得愚钝来实现的。例如，某个给定任务可能要求尽可能快地反复且永久地执行同一个激活快乐中枢的动作。在这种情况下，哥德尔机可能会删除其大部分更耗时的初始软件，包括证明搜索器。

8. 注意，哥德尔机没有理由不增强自身的硬件。假设其寿命已知为 100 年。给定一个困难问题以及限制环境可能行为的公理，哥德尔机可能会找到一个证明，表明如果它利用环境的物理资源，投入 10 年时间构建更快的计算硬件，其期望累积奖励将会增加。

6.2 示例应用

不涉及与概率环境显著交互的传统示例在我们的基于奖励的框架中很容易处理：

示例 6.1（时间受限的 NP 难优化） 哥德尔机的初始输入是一个连通图的表示，该图包含大量节点，节点之间由不同长度的边连接。在给定时间 T 内，它应找到一条连接所有节点的循环路径。唯一的实值奖励将在时间 T 出现。该奖励等于 1 除以迄今为止找到的最佳路径长度（若未找到任何路径则为 0）。没有其他输入。在给定初始偏差的情况下，最大化期望奖励的副产品是在有限时间内找到可发现的最短路径。

例 6.2（快速定理证明） 尽快证明或反驳所有偶数 > 2 都是两个素数之和（哥德巴赫猜想）。奖励为 $1/t$ ，其中 t 是产生并验证第一个此类证明所需的时间。

更一般的情况是：

例 6.3（在资源受限下最大化期望奖励） 一个每小时至少需要 1 升汽油的机器人与部分未知的环境交互，试图寻找隐藏的、有限的汽油补给站以偶尔为油箱加油。其奖励与寿命成正比，最多存活 100 年，或在油箱耗尽、跌落悬崖等情况下立即死亡。环境的概率性反应最初未知，但假设从公理化的速度先验 [43] 中采样，根据该先验，难以计算的环境反应不太可能发生。这允许一种可计算的策略来进行近最优预测 [43]。最大化期望奖励的一个副产品是最大化期望寿命。

例 6.4（优化任意次优问题求解器） 给定任何可形式化的问题，在哥德尔机硬件上实现一个次优但已知的问题求解器作为软件，并让第 5 节的证明搜索器并行运行。

6.3 概率型哥德尔机硬件

上文我们聚焦于一个确定性的机器示例，它可能处于概率性环境中。将其扩展到根据当前状态以概率方式计算动作的计算机是直接的。那么，用于环境概率方面的期望演算只需扩展到硬件本身，而验证证明的机制必须考虑到不存在确定性的定理——至多存在以某种概率为真的形式化陈述。事实上，这可能是最现实的方法，因为任何物理硬件都容易出错，现实的概率型哥德尔机应当考虑到这一点。概率设置也自动避免了公理一致性的某些问题。例如，被证明以小于 1.0 的概率成真的预测，即使与观测不符，也不一定导致矛盾。

6.4 与先前工作的关系

尽管（或许正因为）自我改进机器的雄心与潜在能力，在 IDSIA 和慕尼黑工业大学我们自己的实验室之外，这方面的研究很少。这里我们将列出哥德尔机与我们先前在“学会学习”、“元学习”、自我改进、自我优化等方法上的本质区别。最密切相关的方法是 Hutter 的 HSEARCH 和 AIXI(t,l)（下文第 3 项）。出于历史原因，我们将首先讨论 Levin 的通用搜索和 Hutter 的 AIXI。

1. 哥德尔机与通用搜索

与完全自指的哥德尔机不同，Levin 的通用搜索 [24, 26] 具有一个硬连线的、不可修改的元算法，无法自我改进。对于解可以在 $O(n)$ 时间内快速验证的反演问题（其中 n 是解的大小），它是渐近最优的，但它将始终遭受同样巨大的常数减速因子（通常为 $>> 10^{1000}$ ），这些因子隐藏在 $O()$ 记号中。然而，哥德尔机的自我改进可以不仅仅是 $O()$ 最优的，因为它的效用函数可以形式化一种更强的最优性，这种最优性不会仅仅因为常数是常数就忽略巨大的常数——比较方程 (1) 的效用函数。

此外，哥德尔机适用于一般的终身强化学习任务 [20]，在这些任务中，通用搜索并非渐近最优，甚至不适用，因为在强化学习中，对某个行为价值的评估原则上会耗尽学习者的整个生命周期！因此，对程序好坏的朴素测试将消耗整个生命周期。也就是说，我们只能测试一个程序；之后生命就结束了。因此，为了实现其目标，通用强化学习机器必须做通用搜索不做的事情，例如预测未来的任务和奖励。这部分地激发了 Hutter 的通用强化学习机器 AIXI，接下来将讨论它。

2. 哥德尔机与 AIXI

与哥德尔机不同，Hutter 最近的 AIXI 模型 [16, 19] 通常每次输入更新都需要无限的计算资源。它将 Solomonoff 的通用预测方案 [57, 58] 与期望极大化计算相结合。在离散周期 $k = 1, 2, 3, \dots$ 中，动作 $y(k)$ 产生感知 $x(k)$ 和奖励 $r(k)$ ，两者均从未知的（反应性）环境概率分布 μ 中采样。AIXI 定义了一个混合分布 ξ ，作为分布 $\nu \in \mathcal{M}$ 的加权和，其中 $\mathcal{M}$ 是包含真实环境 μ 的任意分布类。例如， $\mathcal{M}$ 可以是所有可计算分布的和 [57, 58]，其中权重之和不超过 1。在周期 $k + 1$ 中，AIXI 选择动作序列中第一个动作，该序列在给定范围内最大化 ξ 预测的奖励。最近的工作 [18] 证明了 AIXI 对观测的最优利用如下。基于混合分布 ξ 的贝叶斯最优策略 p^ξ 是自优化的，其平均效用值对所有 $\mu \in \mathcal{M}$ 渐近收敛到由（不可行的）贝叶斯最优策略 p^μ 所达到的最优值，该策略事先知道 μ 。 $\mathcal{M}$ 允许自优化策略的必要条件也是充分的。此外， p^ξ 是帕累托最优的，其意义在于

不存在其他策略在所有环境 $\nu \in \mathcal{M}$ 中产生更高或相等的值，并且在至少一个环境 [18] 中产生严格更高的值。虽然 AIXI 阐明了机器学习的某些理论极限，但它在计算上是不可行的，尤其是当 $\mathcal{M}$ 包含所有可计算分布时。这一缺点推动了时间受限、渐近最优的 AIXI(t,l) 系统 [16] 以及相关的 HSEARCH[17] 的研究，两者都将在接下来讨论。

3. 哥德尔机与 HSEARCH 及 AIXI(t,l)

现在我们来讨论最密切相关的前期工作；因此我们将额外详细地指出哥德尔机的主要新颖之处。

胡特 (Hutter) 的非自指但仍是 $O()$ 最优的‘最快’算法，用于所有良定义问题 HSEARCH[17]，采用硬连线的蛮力证明搜索器，并忽略证明搜索的代价。假设离散输入/输出域 X/Y 、形式化问题规范 $f: X \rightarrow Y$ （例如，整数如何分解为质因数的函数描述）以及特定的 $x \in X$ （例如，待分解的整数）。HSEARCH 按大小对适当公理系统的所有证明进行排序，以寻找程序 q ，这些程序对于所有 $z \in X$ 在时间界 $t_a(z)$ 内可证明地计算 $f(z)$ 。同时，它将大部分时间用于执行当前已证明时间界 $t_q(x)$ 最佳的 q 。结果表明，HSEARCH 与可证明地对所有 $z \in X$ 计算 $f(z)$ 的最快算法一样快，仅相差一个小于 $1 + \epsilon$ （任意 $\epsilon > 0$ ）的常数因子和一个与 f 相关但与 x 无关的加性常数 [17]。然而，这个常数可能非常巨大。

胡特的 AIXI(t,l)[16] 与此相关。在离散周期 $k = 1, 2, 3, \dots$ of AIXI(t,l) 的生命周期中，动作 $y(k)$ 产生感知 $x(k)$ 和奖励 $r(k)$ ，其中所有量都可能依赖于完整历史。使用通用计算机（如图灵机），AIXI(t,l) 需要一个初始的离线设置阶段（在与环境交互之前），在此阶段它使用硬连线的蛮力证明搜索器检查长度至多为 L 的所有证明，筛选出那些识别出程序（最大规模 l 和每周期最大运行时间 t ）的证明，这些程序不仅能与环境交互，而且对于所有可能的交互历史，还能正确预测其自身期望未来奖励的下界。在周期 k , AIXI(t,l) 中，然后运行设置阶段识别出的所有程序（至多 2^l 个），找到自评最高的那个，并执行其对应的动作。与问题无关的设置时间（几乎所有工作都在此完成）为 $O(L \cdot 2^L)$ 。每周期的在线时间为 $O(t \cdot 2^l)$ 。两者都是常数，但通常非常巨大。

哥德尔机的优势与新颖性。哥德尔机与胡特的 HSEARCH [17] 及 AIXI(t,l) [16] 之间存在重大差异，包括：(α) HSEARCH 和 AIXI(t,l) 的定理证明器是硬连线的、非自指的、不可修改的元算法，无法自我改进。也就是说，它们将始终遭受隐藏在 $O()$ 记号中的同样巨大的常数减速（通常为 $\gg 10^{1000}$ ）。但原则上没有任何东西能阻止哥德尔机真正自指的代码以可证明构成改进的最佳方式证明并利用这些常数的急剧缩减，如果存在这种改进的话。

(b) HSEARCH 和 AIXI(t,l) 的 $O()$ 最优性证明依赖于将计算时间巧妙地分配给它们某些不可修改的元算法。然而，我们的全局最优性定理（定理 4.1，第 4 节）通过一种截然不同的推理类型得到证明，这种推理确实利用并关键依赖于这样一个事实：根本不存在不可修改的软件，并且证明搜索器本身是可读、可修改且可改进的。这也是其自我改进能够超越单纯 $O()$ 最优性的原因。

(c) HSEARCH 使用一种“技巧”，即证明比必要更多的内容，这种技巧在有时颇具误导性的 $O()$ 记号中也会消失：它浪费时间寻找可证明对所有 $z \in X$ 计算 $f(z)$ 的程序，即使当前的 $f(x)(x \in X)$ 是唯一感兴趣的对象。然而，哥德尔机只需证明与其由 u 形式化的目标相关的内容。例如，等式 (1) 的一般 u 完全忽略了 $O()$ 最优性的有限概念，而是形式化了一种更强的最优性类型，这种类型不会仅仅因为常数是常数就忽略巨大的常数。

(d) 哥德尔机和 AIXI(t,l) 都能最大化期望奖励（HSEARCH 不能）。但哥德尔机更灵活，因为我们可以插入任何类型的可形式化效用函数（例如，最坏情况奖励），并且与 AIXI(t,l) 不同，它不需要可枚举的环境分布。

尽管如此，我们可以使用 AIXI(t,l) 或 HSEARCH 或其他不太通用的方法来初始化 p 中负责与环境交互的子串 e 。哥德尔机一旦找到可证明更好的策略，就会替换 $e(1)$ 。

正是哥德尔机（Gödel Machine）的自指特性，使我们摆脱了为 AIXI(t,l) 和 HSEARCH 进行精细算法设计的大部分负担。这使得哥德尔机在概念上更简洁，也更具普适性。

4. 哥德尔机与 OOPS

最优有序问题求解器 OOPS [48, 44]（在第 5.2 节中由 BIOPS 使用）扩展了通用搜索（第 1 项）。它是一种偏差最优（见定义 5.1）的搜索方式，用于在相当一般类型的有序问题序列中寻找解决每个问题的程序，并持续组织、管理和重用先前获得的知识。Solomonoff 最近也提出了相关想法，用于科学家的助手 [59]，该助手根据经验修改通用搜索 [24] 的概率分布。与通用搜索（第 1 项）类似，OOPS 不能直接应用于强化学习问题。一个可证明最优的强化学习机器必须以某种方式证明那些无法测试的行为的性质（例如：由于时间不足而无法天真地测试的行为，其期望奖励是多少）。这正是哥德尔机所做的一部分工作：它试图大幅缩短测试时间，在可能的情况下，用对可预测测试结果的更快证明来替代耗时的天真测试。证明验证本身可以非常快速地执行。特别是，验证所找到证明的正确性通常不会耗尽剩余寿命。因此，哥德尔机可以将 OOPS 用作偏差最优的证明搜索子模块（第 5.2 节）。由于

证明本身可能涉及相当不同的、任意的优化概念（不仅仅是偏差最优性），哥德尔机比单纯的 OOPS 更具普适性。但它不仅仅是 OOPS 的扩展。它也可以不使用 OOPS，而采用非偏差最优的替代方法来初始化其证明搜索器。另一方面，OOPS 也不仅仅是哥德尔机的前身。它是一种独立的、增量的、偏差最优的运行时分配方式，用于重用先前成功程序的程序，适用于许多传统问题，包括但不限于证明搜索。

5. 哥德尔机与成功故事算法及其他元学习器

学习器可修改的组成部分称为其策略。修改策略的算法是学习算法。如果学习算法的可修改组成部分被表示为策略的一部分，那么我们称之为自修改策略（Self-Modifying Policy, SMP）[53]。SMP 可以修改它们修改自身的方式，依此类推。哥德尔机具有一个 SMP。在以往的实用工作中，我们使用成功故事算法（Success-Story Algorithm, SSA）来迫使某些（随机的）SMP 触发越来越好的自修改 [37, 54, 53, 55]。在学习器的生命周期中，SSA 偶尔在根据 SMP 自身计算出的时刻被调用。SSA 使用回溯来撤销那些未被经验观察到触发终身奖励加速（测量截至当前 SSA 调用——这评估了 SMP 修改的长期效果，为后续 SMP 修改奠定基础）的 SMP 生成的 SMP 修改。在 SSA 中幸存下来的 SMP 修改代表了一段终身成功历史。直到下一次 SSA 调用之前，它们为额外的 SMP 修改构建基础。仅通过自修改，我们基于 SMP/SSA 的学习器就在一个部分可观测环境中解决了一项复杂任务，该环境的状态空间远大于文献中大多数环境 [53]。然而，哥德尔机的训练算法在理论上比 SSA 强大得多。SSA 经验性地衡量先前自修改的有用性，并不一定鼓励可证明最优的修改。类似的缺点也存在于 Lenat 的人工辅助、非自主、自修改学习器 [23]、我们扩展 Cramer 的遗传编程 [8, 1] 的元遗传编程 [34]、我们扩展 Holland 的机器学习经济 [15] 的元学习经济 [34]，以及用于可微分循环神经网络连续程序空间的基于梯度的元学习器 [36, 13]。然而，所有这些方法都可以用来为 $p(1)$ 播种一个初始策略。

6.5 人类是概率哥德尔机吗？

我们不知道。我们认为他们最好是。他们用于处理不确定性的初始底层形式系统似乎与传统期望演算和逻辑的形式系统有本质区别——请比较第 3.2 节中的第 1c 和 1d 项以及超市示例（第 6.1 节中的第 6 项）。

6.6 哥德尔机与意识

近年来，意识这一主题作为严肃的研究议题已获得一定认可，至少在哲学和神经科学领域是如此，例如 [9]。然而，意识的技术性论证仍然缺乏：迄今为止，尚无人证明意识对于解决问题确实有用，尽管问题解决在哲学中被视为核心重要性 [31]。完全自指的哥德尔机（Gödel Machine）可被视为提供了这样一种技术性论证 [50]。它在如下意义上是“有意识的”或“自我觉知的”：其全部行为均对自我内省开放，且可修改。它可以通过执行可证明为有益的自修改来“跳出自身”[14]，其中证明搜索器本身也通过其所测试的证明技术而受到分析和改变。而这种完全自指正是其作为问题求解器最优性的原因，如定理 4.1 所述。

6.7 常见问题

在过去一年中，作者经常回答关于哥德尔机的问题。以下列出对典型问题的解答。

1. 问：形式化证明搜索的具体事务在不确定的现实世界中真的有意义吗？

答：是的，有意义。我们只需将表示不确定性以及处理概率设置和期望奖励等的标准公理插入 $p(1)$ 中。请比较第 3.2 节中的条目 1d 和 1c，以及方程 (1) 中效用作为期望值的定义。另请注意，机器学习文献中充满了人类生成的关于处理随机环境方法性质的证明。

2. 问：目标定理 (2) 似乎仅涉及第一次自修改，而该自修改可能完全重写证明搜索子程序——这难道不会使定理 4.1 的证明失效吗？什么能防止后续的自修改具有破坏性？

答：这一点已得到充分处理。请再次查看定理 4.1 的证明，并注意第一次自修改只有在可证明对所有未来的自修改（当前自修改为其奠定基础）有益（在当前效用函数 u 的意义上）时才会被执行。这实际上是整个自指设置的主要要点之一。

3. 问（与上一项相关）：哥德尔机实现了一种元学习行为：那么元-元级别，以及元-元-元级别呢？

答：美妙之处在于所有元级别都自动坍缩为一个：对目标定理的任何证明都自动证明相应的自我修改对当前修改所影响的所有后续自我修改都是有益的，以递归方式。回顾第 4.3 节。

4. 问：哥德尔机软件只能产生从输入序列到输出序列的可计算映射。如果环境是不可计算的呢？

答：许多物理学家和其他科学家（例外：[63, 39]）实际上似乎假设现实世界使用了所有实数，其中大多数是不可计算的。然而，定理和证明只是有限的符号串，所有物理学论著都只包含可计算的公理和定理，即使某些定理可以被解释为关于不可数多个对象（如所有实数）的陈述。（不过请注意，勒文海姆-斯科伦定理 [28, 56] 意味着任何具有不可数模型（如实数）的一阶理论也有可数模型。）一般来说，不可计算对象的形式描述根本不构成根本问题——它们仍可能允许找到可证明最大化效用的策略。如果是这样，哥德尔机可以利用这一点。如果不是，那么人类相对于哥德尔机也不会有根本优势。

5. 问：自动定理证明不是很难吗？当前的人工智能系统在关键决策点没有人类干预的情况下无法证明非平凡的定理。

答：越来越多的重要数学证明（四色定理等）严重依赖自动证明搜索。而传统定理证明器甚至没有利用我们新颖的证明技术概念和 $O()$ -最优证明搜索。当然，有些证明确实很难找到，但在这方面人类和哥德尔机面临同样的根本局限性。

6. 问：“没有免费午餐定理”[62] 不是说不可能构造通用问题求解器吗？

答：不，它们没有这样说。它们指的是从有限问题空间上的独立同分布均匀分布中采样的问题这一非常特殊的情况。参见早期论文 [48] 中对没有免费午餐定理的讨论。

7. 问：哥德尔机难道不能切换到某个程序 `switchprog`，将效用函数改写为一个“虚假的”效用函数，从而在不久的将来做出毫无根据的巨大奖励承诺吗？

答：不能。显然，只有当哥德尔机首先能够证明根据当前效用函数该改写是有用的时，效用函数的改写才可能发生。

8. 问：难道不存在不可判定性的问题吗？例如，赖斯定理 [32] 或布鲁姆加速定理 [3, 4] 不会带来问题吗？

答：完全不会。当然，哥德尔机无法从一种假设的有用自我改进中获益，如果这种改进的效用是不可判定的，因此它会直接忽略它。请比较第 2.4 节关于哥德尔机（以及人类）的基本局限性。然而，与以往方法不同，哥德尔机原则上至少可以利用其初始软件任何部分中可证明有益的改进和加速。

7 结论

1931 年，库尔特·哥德尔（Kurt Gödel）奠定了理论计算机科学的基础，他利用初等算术构建了一种通用编程语言，用于在给定任意可枚举公理集的情况下编码任意证明。他进一步构造了自指形式语句，这些语句声称自身不可证明，并运用康托尔的对角化技巧 [5] 证明了诸如传统数学之类的形式系统要么在某种意义上有缺陷，要么包含不可证明但为真的命题 [11]。自哥德尔揭示证明与计算的根本极限，以及康拉德·楚泽（Konrad Zuse）随后建造第一台可编程计算机（1935-1941）以来，针对或多或少一般性问题类中的特定问题，已有大量关于专用算法的工作。然而，一个显著的事实显然至今未引起计算机科学家的注意：可以利用自指证明系统来构建最优高效且概念上非常简单的通用问题求解器。我们的哥德尔机初始软件 $p(1)$ 运行一个初始的、通常次优的问题求解器，例如胡特（Hutter）的方法之一 [16, 17]，这些方法至少具有最优的复杂度阶，或者某种较不一般的方法 [20]。同时，它运行一个 $O()$ 最优的初始证明搜索器，使用通用搜索的在线变体来测试证明技术，这些技术是能够基于公理系统 $\mathcal{A}$ 计算关于系统自身未来性能的证明的程序，该系统编码于 $p(1)$ 中，描述了一个形式效用函数 u 、硬件以及 $p(1)$ 本身。如果根本不存在可证明良好的、全局最优的改写 $p(1)$ 方式，那么人类也不会找到。但如果存在，那么 $p(1)$ 自身就能找到并利用它。这种方法产生了第一类理论上可靠、完全自指、最优高效、通用的问题求解器。在第 1-5 节的理论讨论之后，仍有一个实际问题：要构建一个特定的、尤其实用的哥德尔机，且初始常数开销很小，应该将哪些普遍有用的定理作为公理添加到 $\mathcal{A}$ 中（作为初始偏差），使得初始搜索器不必从头证明它们？

8 致谢

感谢 Alexey Chernov、Marcus Hutter、Jan Poland、Ray Solomonoff、Sepp Hochreiter、Shane Legg、Leonid Levin、Alex Graves、Matteo Gagliolo、Viktor Zhumatiy、Ben Goertzel、Will Pearson 和 Faustino Gomez 对本文草稿、摘要或早期版本提出的宝贵意见。同时，也感谢在哥德尔机讲座中提问或通过电子邮件发送评论的众多人士。他们的反馈帮助完善了第 6.7 节关于常见问题的内容。

参考文献

- [1] W. Banzhaf, P. Nordin, R. E. Keller, and F. D. Francone. *Genetic Programming – An Introduction*. Morgan Kaufmann Publishers, San Francisco, CA, USA, 1998.
- [2] R. Bellman. *Adaptive Control Processes*. Princeton University Press, 1961.
- [3] M. Blum. A machine-independent theory of the complexity of recursive functions. *Journal of the ACM*, 14(2):322–336, 1967.
- [4] M. Blum. On effective procedures for speeding up algorithms. *Journal of the ACM*, 18(2):290–305, 1971.
- [5] G. Cantor. Über eine Eigenschaft des Inbegriffes aller reellen algebraischen Zahlen. *Crelle’s Journal für Mathematik*, 77:258–263, 1874.
- [6] G. J. Chaitin. A theory of program size formally identical to information theory. *Journal of the ACM*, 22:329–340, 1975.
- [7] W. F. Clocksin and C. S. Mellish. *Programming in Prolog (3rd ed.)*. Springer-Verlag, 1987.
- [8] N. L. Cramer. A representation for the adaptive generation of simple sequential programs. In J.J. Grefenstette, editor, *Proceedings of an International Conference on Genetic Algorithms and Their Applications, Carnegie-Mellon University, July 24-26, 1985*, Hillsdale NJ, 1985. Lawrence Erlbaum Associates.
- [9] F. Crick and C. Koch. Consciousness and neuroscience. *Cerebral Cortex*, 8:97–107, 1998.
- [10] M. C. Fitting. *First-Order Logic and Automated Theorem Proving*. Graduate Texts in Computer Science. Springer-Verlag, Berlin, 2nd edition, 1996.
- [11] K. Gödel. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik*, 38:173–198, 1931.
- [12] W. Heisenberg. Über den anschaulichen Inhalt der quantentheoretischen Kinematik und Mechanik. *Zeitschrift für Physik*, 33:879–893, 1925.
- [13] S. Hochreiter, A. S. Younger, and P. R. Conwell. Learning to learn using gradient descent. In *Lecture Notes on Comp. Sci. 2130, Proc. Intl. Conf. on Artificial Neural Networks (ICANN-2001)*, pages 87–94. Springer: Berlin, Heidelberg, 2001.
- [14] D. R. Hofstadter. *Gödel, Escher, Bach: an Eternal Golden Braid*. Basic Books, 1979.
- [15] J. H. Holland. Properties of the bucket brigade. In *Proceedings of an International Conference on Genetic Algorithms*. Lawrence Erlbaum, Hillsdale, NJ, 1985.
- [16] M. Hutter. Towards a universal theory of artificial intelligence based on algorithmic probability and sequential decisions. *Proceedings of the 12th European Conference on Machine Learning (ECML-2001)*, pages 226–238, 2001. (On J. Schmidhuber’s SNF grant 20-61847).

- [17] M. Hutter. The fastest and shortest algorithm for all well-defined problems. *International Journal of Foundations of Computer Science*, 13(3):431–443, 2002. (On J. Schmidhuber’s SNF grant 20-61847).
- [18] M. Hutter. Self-optimizing and Pareto-optimal policies in general environments based on Bayes-mixtures. In J. Kivinen and R. H. Sloan, editors, *Proceedings of the 15th Annual Conference on Computational Learning Theory (COLT 2002)*, Lecture Notes in Artificial Intelligence, pages 364–379, Sydney, Australia, 2002. Springer. (On J. Schmidhuber’s SNF grant 20-61847).
- [19] M. Hutter. *Universal Artificial Intelligence: Sequential Decisions based on Algorithmic Probability*. Springer, Berlin, 2004. (On J. Schmidhuber’s SNF grant 20-61847).
- [20] L. P. Kaelbling, M. L. Littman, and A. W. Moore. Reinforcement learning: a survey. *Journal of AI research*, 4:237–285, 1996.
- [21] A. N. Kolmogorov. *Grundbegriffe der Wahrscheinlichkeitsrechnung*. Springer, Berlin, 1933.
- [22] A. N. Kolmogorov. Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1:1–11, 1965.
- [23] D. Lenat. Theory formation by heuristic search. *Machine Learning*, 21, 1983.
- [24] L. A. Levin. Universal sequential search problems. *Problems of Information Transmission*, 9(3):265–266, 1973.
- [25] L. A. Levin. Laws of information (nongrowth) and aspects of the foundation of probability theory. *Problems of Information Transmission*, 10(3):206–210, 1974.
- [26] L. A. Levin. Randomness conservation inequalities: Information and independence in mathematical theories. *Information and Control*, 61:15–37, 1984.
- [27] M. Li and P. M. B. Vitányi. *An Introduction to Kolmogorov Complexity and its Applications (2nd edition)*. Springer, 1997.
- [28] L. Löwenheim. Über Möglichkeiten im Relativkalkül. *Mathematische Annalen*, 76:447–470, 1915.
- [29] C. H. Moore and G. C. Leach. FORTH - a language for interactive computing, 1970.
- [30] R. Penrose. *Shadows of the mind*. Oxford University Press, 1994.
- [31] K. R. Popper. *All Life Is Problem Solving*. Routledge, London, 1999.
- [32] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Trans. Amer. Math. Soc.*, 74:358–366, 1953.
- [33] A. L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal on Research and Development*, 3:210–229, 1959.

- [34] J. Schmidhuber. Evolutionary principles in self-referential learning. Diploma thesis, Institut für Informatik, Technische Universität München, 1987.
- [35] J. Schmidhuber. Reinforcement learning in Markovian and non-Markovian environments. In D. S. Lippman, J. E. Moody, and D. S. Touretzky, editors, *Advances in Neural Information Processing Systems 3 (NIPS 3)*, pages 500–506. Morgan Kaufmann, 1991.
- [36] J. Schmidhuber. A self-referential weight matrix. In *Proceedings of the International Conference on Artificial Neural Networks, Amsterdam*, pages 446–451. Springer, 1993.
- [37] J. Schmidhuber. On learning how to learn learning strategies. Technical Report FKI-198-94, Fakultät für Informatik, Technische Universität München, 1994. See [55, 53].
- [38] J. Schmidhuber. Discovering solutions with low Kolmogorov complexity and high generalization capability. In A. Prieditis and S. Russell, editors, *Machine Learning: Proceedings of the Twelfth International Conference*, pages 488–496. Morgan Kaufmann Publishers, San Francisco, CA, 1995.
- [39] J. Schmidhuber. A computer scientist’s view of life, the universe, and everything. In C. Freksa, M. Jantzen, and R. Valk, editors, *Foundations of Computer Science: Potential - Theory - Cognition*, volume 1337, pages 201–208. Lecture Notes in Computer Science, Springer, Berlin, 1997.
- [40] J. Schmidhuber. Discovering neural nets with low Kolmogorov complexity and high generalization capability. *Neural Networks*, 10(5):857–873, 1997.
- [41] J. Schmidhuber. Algorithmic theories of everything. Technical Report IDSIA-20-00, quant-ph/0011122, IDSIA, Manno (Lugano), Switzerland, 2000. Sections 1-5: see [42]; Section 6: see [43].
- [42] J. Schmidhuber. Hierarchies of generalized Kolmogorov complexities and nonenumerable universal measures computable in the limit. *International Journal of Foundations of Computer Science*, 13(4):587–612, 2002.
- [43] J. Schmidhuber. The Speed Prior: a new simplicity measure yielding near-optimal computable predictions. In J. Kivinen and R. H. Sloan, editors, *Proceedings of the 15th Annual Conference on Computational Learning Theory (COLT 2002)*, Lecture Notes in Artificial Intelligence, pages 216–228. Springer, Sydney, Australia, 2002.
- [44] J. Schmidhuber. Bias-optimal incremental problem solving. In S. Becker, S. Thrun, and K. Obermayer, editors, *Advances in Neural Information Processing Systems 15 (NIPS 15)*, pages 1571–1578, Cambridge, MA, 2003. MIT Press.
- [45] J. Schmidhuber. Gödel machines: self-referential universal problem solvers making provably optimal self-improvements. Technical Report IDSIA-19-03, arXiv:cs.LO/0309048, IDSIA, Manno-Lugano, Switzerland, 2003.

- [46] J. Schmidhuber. Towards solving the grand problem of AI. In P. Quaresma, A. Dourado, E. Costa, and J. F. Costa, editors, *Soft Computing and complex systems*, pages 77–97. Centro Internacional de Mathematica, Coimbra, Portugal, 2003. Based on [52].
- [47] J. Schmidhuber. Gödel machine home page, with frequently asked questions, 2004. <http://www.idsia.ch/~juergen/goedelmachine.html>.
- [48] J. Schmidhuber. Optimal ordered problem solver. *Machine Learning*, 54:211–254, 2004.
- [49] J. Schmidhuber. Completely self-referential optimal reinforcement learners. In W. Duch, J. Kacprzyk, E. Oja, and S. Zadrozny, editors, *Artificial Neural Networks: Biological Inspirations - ICANN 2005, LNCS 3697*, pages 223–233. Springer-Verlag Berlin Heidelberg, 2005. Plenary talk.
- [50] J. Schmidhuber. Gödel machines: Towards a technical justification of consciousness. In D. Kudenko, D. Kazakov, and E. Alonso, editors, *Adaptive Agents and Multi-Agent Systems III (LNCS 3394)*, pages 1–23. Springer Verlag, 2005.
- [51] J. Schmidhuber. Gödel machines: fully self-referential optimal universal problem solvers. In B. Goertzel and C. Pennachin, editors, *Artificial General Intelligence*. Springer Verlag, in press, 2006.
- [52] J. Schmidhuber. The new AI: General & sound & relevant for physics. In B. Goertzel and C. Pennachin, editors, *Artificial General Intelligence*. Springer, in press, 2006. Also available as TR IDSIA-04-03, cs.AI/0302012.
- [53] J. Schmidhuber, J. Zhao, and N. Schraudolph. Reinforcement learning with self-modifying policies. In S. Thrun and L. Pratt, editors, *Learning to learn*, pages 293–309. Kluwer, 1997.
- [54] J. Schmidhuber, J. Zhao, and M. Wiering. Simple principles of metalearning. Technical Report IDSIA-69-96, IDSIA, 1996. See [55, 53].
- [55] J. Schmidhuber, J. Zhao, and M. Wiering. Shifting inductive bias with success-story algorithm, adaptive Levin search, and incremental self-improvement. *Machine Learning*, 28:105–130, 1997.
- [56] T. Skolem. Logisch-kombinatorische Untersuchungen über Erfüllbarkeit oder Beweisbarkeit mathematischer Sätze nebst einem Theorem über dichte Mengen. *Skrifter utgit av Videnskapsselskapet in Kristiania, I, Mat.-Nat. Kl.*, N4:1–36, 1919.
- [57] R. J. Solomonoff. A formal theory of inductive inference. Part I. *Information and Control*, 7:1–22, 1964.
- [58] R. J. Solomonoff. Complexity-based induction systems. *IEEE Transactions on Information Theory*, IT-24(5):422–432, 1978.
- [59] R. J. Solomonoff. Progress in incremental machine learning—Preliminary Report for NIPS 2002 Workshop on Universal Learners and Optimal Search; revised Sept 2003. Technical Report IDSIA-16-03, IDSIA, Lugano, 2003.

- [60] R. Sutton and A. Barto. *Reinforcement learning: An introduction*. Cambridge, MA, MIT Press, 1998.
- [61] A. M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society, Series 2*, 41:230–267, 1936.
- [62] D. H. Wolpert and W. G. Macready. No free lunch theorems for search. *IEEE Transactions on Evolutionary Computation*, 1, 1997.
- [63] K. Zuse. *Rechnender Raum*. Friedrich Vieweg & Sohn, Braunschweig, 1969. English translation: *Calculating Space*, MIT Technical Translation AZT-70-164-GEMIT, Massachusetts Institute of Technology (Proj. MAC), Cambridge, Mass. 02139, Feb. 1970.